Program Overview:

My program creates a 9x9 Sudoku board by first creating a grid/board using a 2D array then creating methods that can be used to create the random numbers in the board and formatting them. First, a method is created to change the numbers in a given array by swapping the numbers in the array randomly. Then, a method that creates a 1D array with numbers 1-9 is created and assigns those values to each plot in the 2D array with the help of the first method that we created. The method loops through the whole 2D array so each plot is given a value. Then, another method is created to format and organize the Sudoku board. Lastly the main method creates a 2D array and calls on the second and third method that we made.

Data Structures:

I used a 2D array by making my Sudoku board based on numbers of a 2D array called "grid"(int[][] grid) with each value in the grid corresponding to a number in the Sudoku board, with grid[0][0] being the first number and grid[8][8] being the last number. It is used in the methods "CompleteBoard" and "printBoard" to fill the Sudoku board and print it out. I used an 1D array(intital[][]) as a template to change and randomize the numbers for each row of the 2D array in the method "ChangeNums".

Algorithm Design:

I used a patterned based construction method to create the Sudoku board. I used the method "ChangeNums" to randomize each number in a template 1D by swapping each number in an array. A method "CompleteBoard" was created so I can create a template array called "initial[]" with numbers 1-9 and call the method "ChangeNums" to randomize it. Then I used a

nested for-loop so that each 3 rows are the same 9 numbers but shifted 3 numbers to the right. Then, you add 1 shift to the right for every 3 rows so that all the rows are different because there is an additional shift. The numbers are then entered into grid[][]. The same order of numbers are used for every row but shifted which makes sure that a number in the Sudoku board does not have the same number as another in the same row, column, or 3x3 plot.

Methods:

- **Main method:** This method is used to create the board/2D array grid[9][9] to store each Sudoku board value and to call the "CompleteBoard" and "printBoard" methods to fill in each number for the board and create the board.
- **ChangeNums:** This method creates a new instance of the java.util.Random; calls by using the statement Random r = new Random();. Then uses a for-loop to run through every number in a 1D array and randomly swapping its place with another number in the array.
- **CompleteBoard:** This method creates a 1D array called "initial[]" that has values 1-9 and then calls the method "ChangeNums" to randomize the number in the array so the numbers are 1-9 but in different order. Then a nested for-loop is used to shift the values in each row to the right based on the column position of the number added to change(which is taken from the line "int change = (i * 3 + i / 3) % 9") as the row changes. This value is then added to a specific slot in the grid[][] 2D array.
- **printBoard:** This method creates the Sudoku board using nested for-loops so that it creates borders and separations along with the grid[][] values so that the Sudoku board is created with nice borders to separate the numbers and plots. The horizontal borders are

created by using the if-statement “if (i % 3 == 0)” which creates the borders before the values and after every 3 rows because i represent the number of rows. Then the vertical borders are created by using the if-statement “if (j % 3 == 0)” which creates a border before the values horizontally and every after every 3 columns. The grid value is printed out in the for-loop “for (int j = 0; j < 9; j++)” which also contains the if-statement for the vertical borders. Then, after each value in a row is printed, “| “ is printed to close the border and after all the values in the grid are printed out, the horizontal border is printed again.

Testing Plan

I tested my program by running the program multiple times and choosing a random number, checking the horizontal, vertical, and 3x3 plot that it is in so that there is not another number that is the same as that one.

Challenges and Solutions

One challenge that I solved was making sure that the board passed the horizontal, vertical and 3x3 plot test. I did this by deriving a formula which can be seen from the line “int change = (i * 3 + i / 3) % 9;”. This lets each row be a shifted version of the first row. The “i/3” adds 1 for each 3 rows so the first, fourth, and seventh row are not the same. For example, the second row is 3 values shifted to the right from the first row, the third row is 6 values shifted to the right from the first row, but the fourth row is only 1 value shifted to the right from the first row. An

example Sudoku created from this code is shown below.

```
+-------+-------+-------+
| 1 9 7 | 4 6 3 | 8 2 5 |
| 4 6 3 | 8 2 5 | 1 9 7 |
| 8 2 5 | 1 9 7 | 4 6 3 |
+-------+-------+-------+
| 9 7 4 | 6 3 8 | 2 5 1 |
| 6 3 8 | 2 5 1 | 9 7 4 |
| 2 5 1 | 9 7 4 | 6 3 8 |
+-------+-------+-------+
| 7 4 6 | 3 8 2 | 5 1 9 |
| 3 8 2 | 5 1 9 | 7 4 6 |
| 5 1 9 | 7 4 6 | 3 8 2 |
+-------+-------+-------+
```